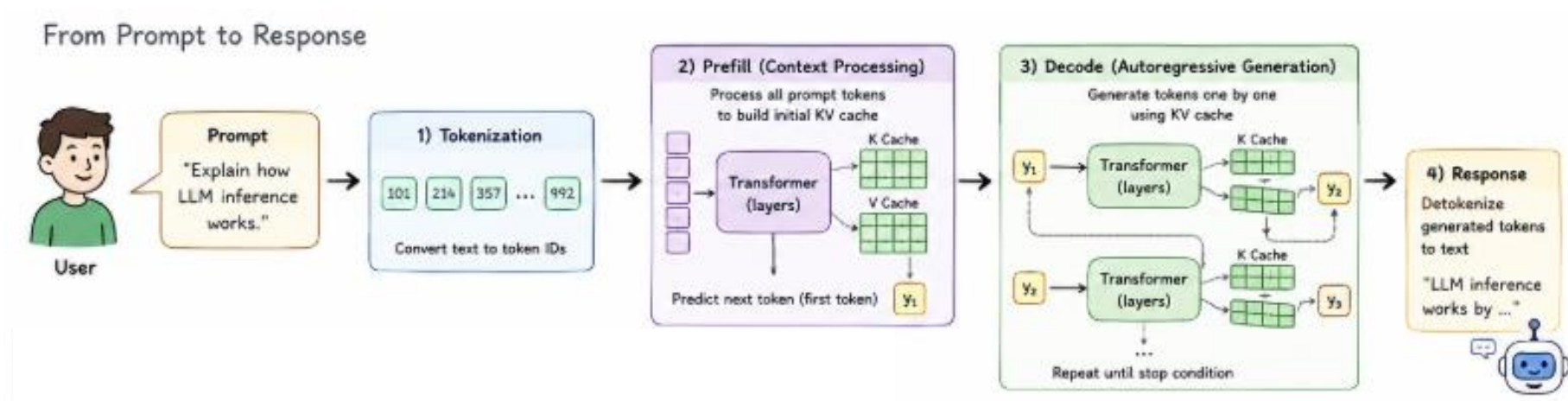
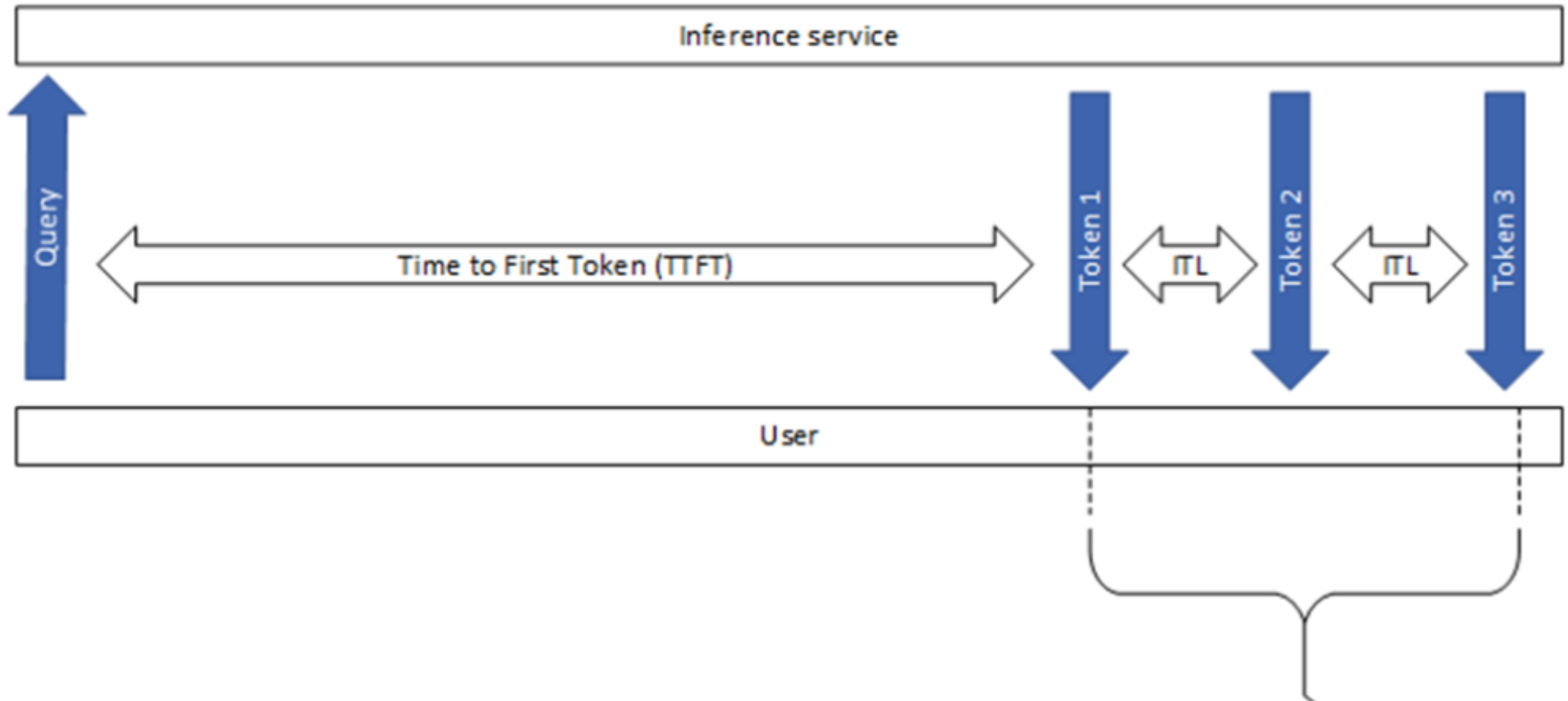


Benchmarking

LLM Pipeline



Metrics



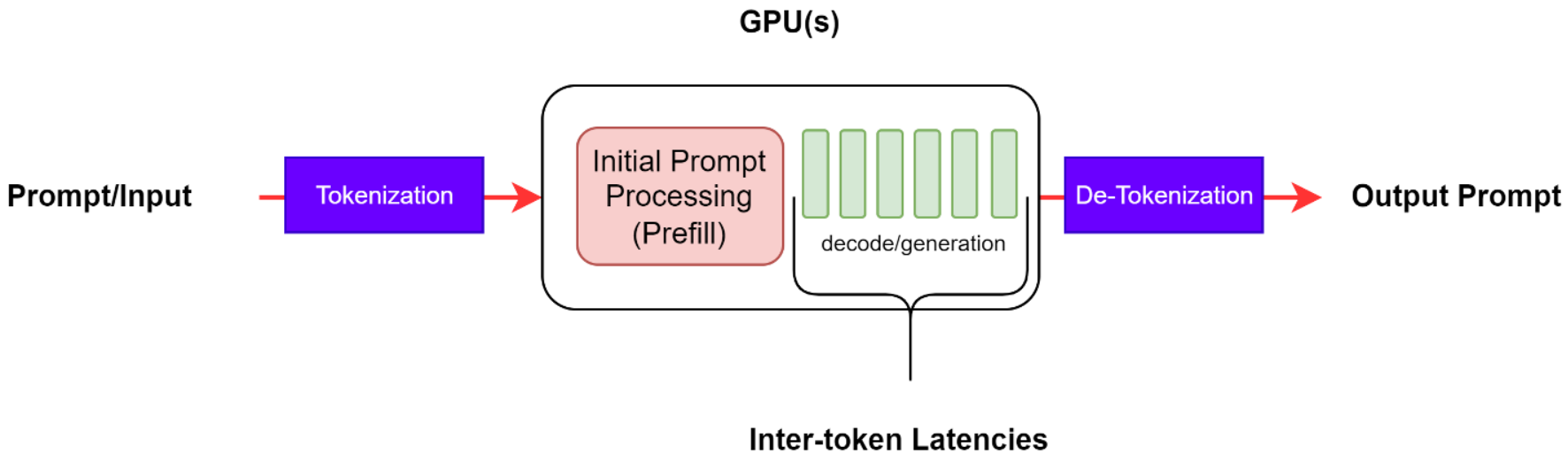
Metrics -- Time to First Token (TTFT)

This metric shows how long a user needs to wait before seeing the model's output. This is the time it takes from submitting the query to receiving the first token (if the response is not empty).

Time to first token generally includes both request queuing time, prefill time and network latency. The longer the prompt, the larger the TTFT. This is because the attention mechanism requires the whole input sequence to compute and create the so-called key-value cache (aka.KV-cache), from which point the iterative generation loop can begin. Additionally, a production application can have several requests in progress, therefore one request's prefill phase may overlap with another request's generation phase.

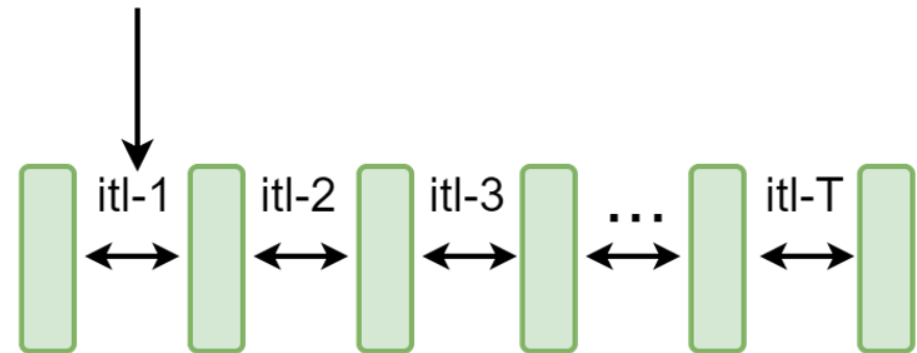
- request queuing time
- network overhead
- tokenization time
- prefill computation time
- first-token sampling / selection time
- scheduler overhead

Metrics -- Time Per Output Token (TPOT)



model size
decode batch size
KV cache length
memory bandwidth
GPU utilization
sampling strategy
scheduler overhead
concurrency

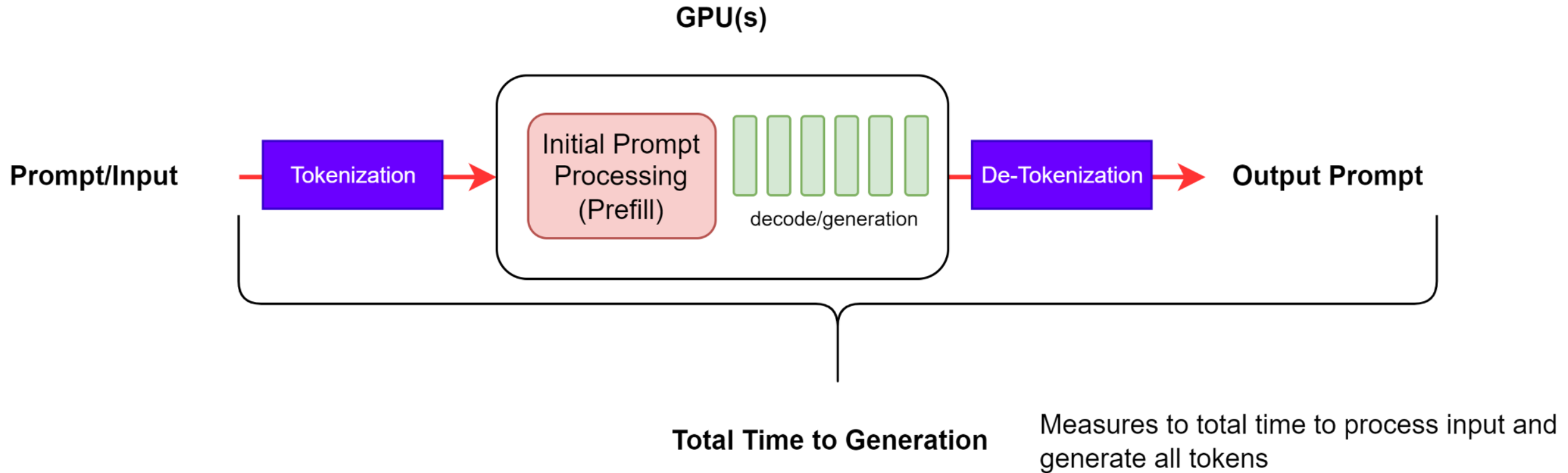
Inter-token Latencies - Should be constant over all token generations. Constant time implies efficient generation



Metrics -- Tokens Per Second (TPS) and Requests/sec



Metrics -- End-to-End Request Latency (e2e_latency)



Metrics – P95

What is P95 Latency?





Average latency can hide slow requests

A few very slow requests can inflate the average—or get lost in it.

VS

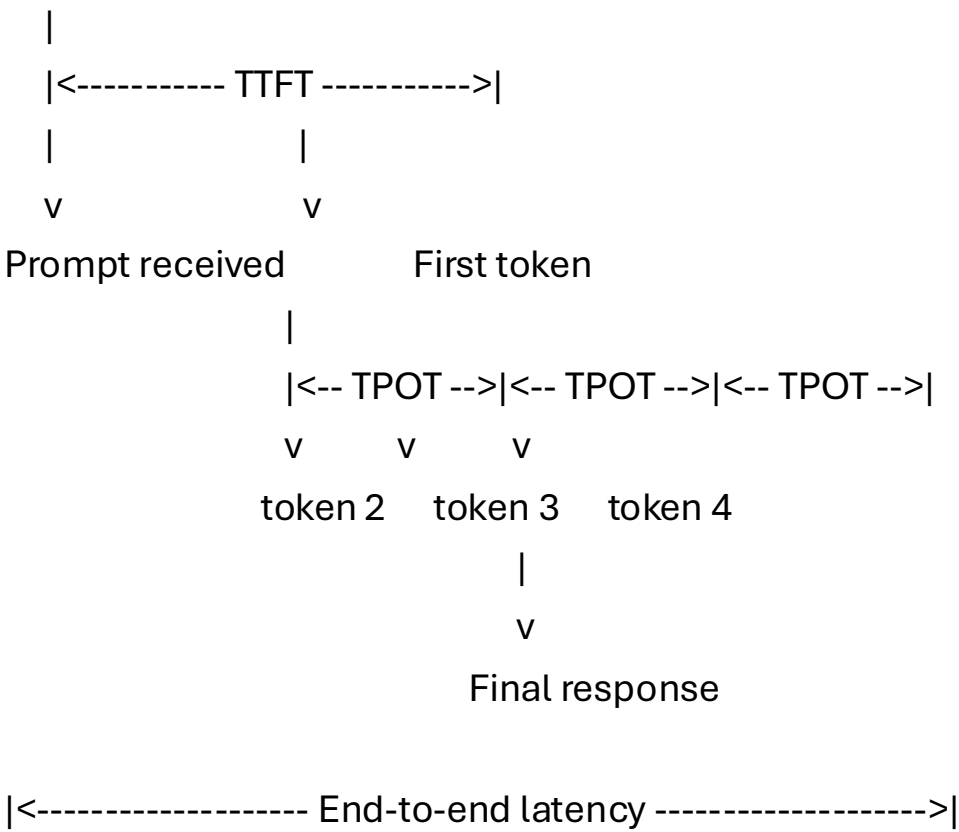


P95 reveals tail latency

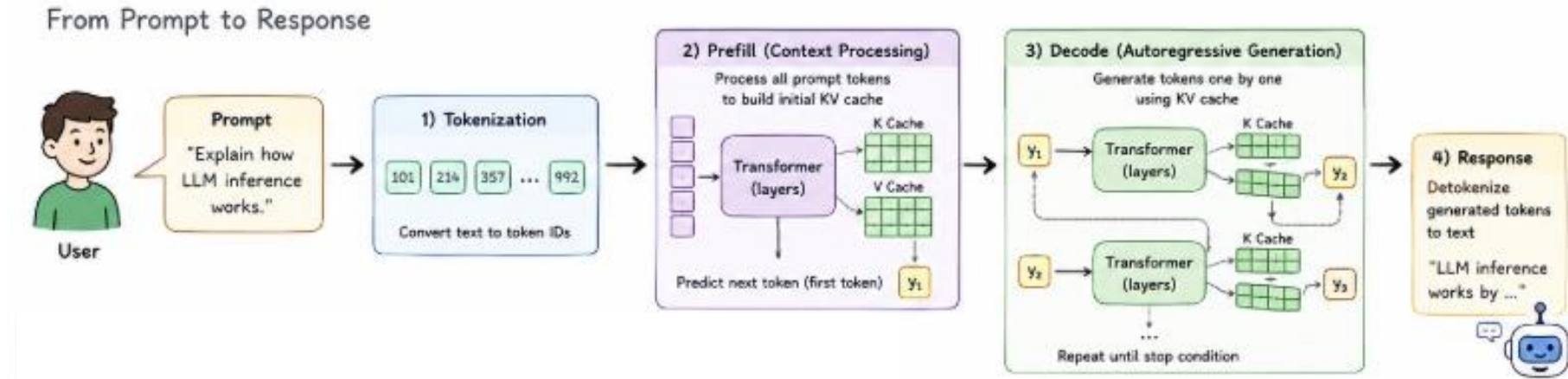
It focuses on the experience of the typical user and exposes the slow tail.

Metrics -- summary

Request starts



Prefill



Prefill processes the full input prompt and builds the initial KV cache before generating the first token.

Prefill

Prompt tokens: $x_1, x_2, x_3, \dots, x_N$

|
v

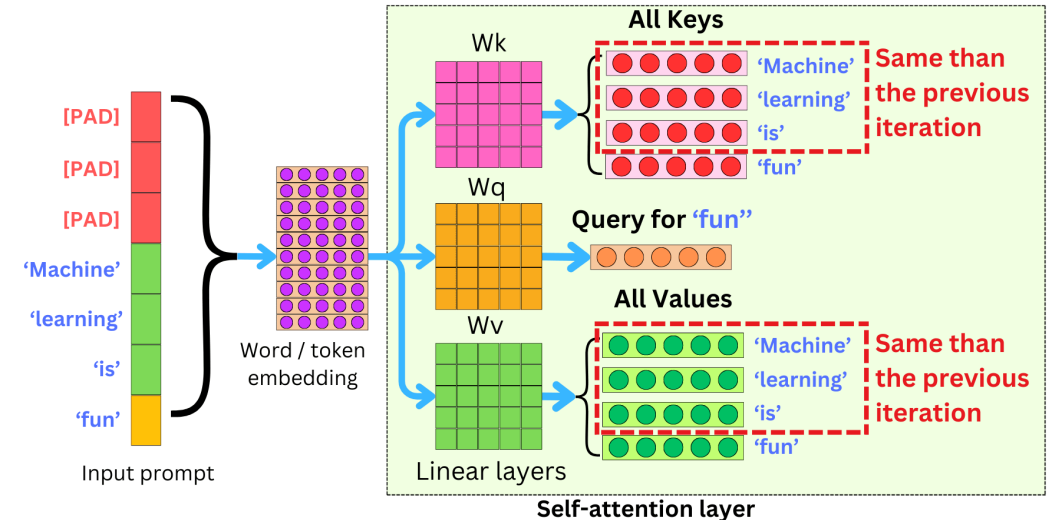
Prefill

- process full prompt
- compute Q/K/V for prompt
- build initial KV cache
- generate first token y_1

|
v

- TTFT is the time from the start of a request until the first token is generated.
- It mainly reflects the cost of the prefill stage and the system's responsiveness.

Longer prompt $\rightarrow$ more prefill computation (initial KV cache) $\rightarrow$ larger attention cost $\rightarrow$ higher TTFT



Example:

prompt 32 tokens, prefill 32 tokens

prompt 1024 tokens, prefill 1024 tokens

Prefill

Prompt tokens: $x_1, x_2, x_3, \dots, x_N$

|
v

Prefill

- process full prompt
- compute Q/K/V for prompt
- build initial KV cache
- generate first token y_1

|
v

What is the bottleneck of prefill?

- compute
- large attention computation
- matrix multiplication
- activation memory
- prefill batch size

Decode

First token y_1 + KV cache

|
v

Decode step 1

- input y_1
- read cached K/V
- generate y_2
- append new K/V

|
v

Decode step 2 $\rightarrow y_3$

|
v

Decode step 3 $\rightarrow y_4$

- TPOT is the average time required to generate each token after the first token.
- It mainly reflects the sustained generation speed of the decode stage.

Unlike prefill, decode is sequential across output tokens.

Decode is memory-bound: since input and output need HBM writing and read(KV cache and y_{n-1})

Difference between prefill and decode

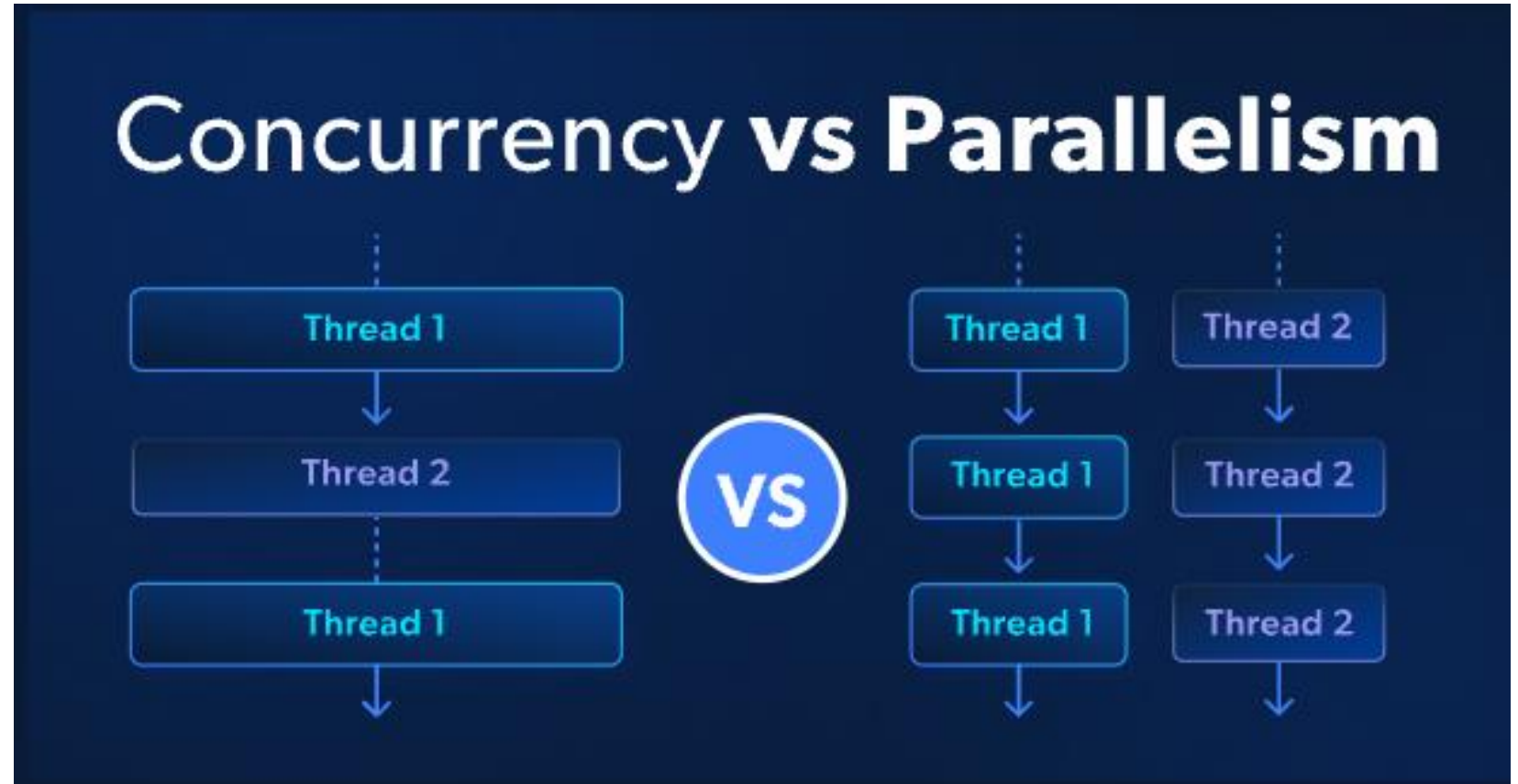
Aspect	Prefill	Decode 
Main purpose	Process the full input prompt before generation starts	Generate output tokens one by one
When it happens	Before the first output token	After the first output token
Input	All prompt tokens at once	The latest generated token plus KV cache
Output	First output token and initial KV cache	Next output token and updated KV cache
Related metric	Mainly affects TTFT	Mainly affects TPOT and total generation latency
Parallelism	Highly parallel across prompt tokens	Sequential across output tokens
Computation pattern	Large matrix operations over the full prompt	Repeated small steps, one token at a time
Bottleneck tendency	More compute-bound	More memory-bound

Why compute-heavy?	The model processes many prompt tokens together through attention and MLP layers	Each step processes only one new token
Why memory-sensitive?	Builds the initial KV cache	Repeatedly reads and appends to the KV cache
Effect of prompt length	Longer prompt makes prefill slower and increases TTFT	Longer prompt also increases the KV cache that decode must attend to
Effect of output length	Not directly affected by output length	Longer output means more decode steps
GPU utilization	Usually better because the work is larger and more parallel	Can be lower for small batch sizes because each step is small
In simple words	"Read and understand the whole prompt first"	"Continue writing the answer token by token"

concurrency

Concurrency = Multiple tasks are in progress at the same time.

Parallelism = Multiple tasks are physically executed at the exact same time.



concurrency

Prefill:

1. Queueing Delay:

request arrives -> waits in queue -> prefill starts -> first token

$TTFT = \text{queueing delay} + \text{prefill time} + \text{first-token overhead}$

2. Prefill Contention

Multiple requests may need to run prefill at the same time.

Prefill is compute-heavy because it processes the full prompt using large matrix operations.

3. Batching Delay

Some serving systems intentionally wait for a short time to collect more requests into a batch in order to improve throughput.

Decode:

Concurrency can create batching opportunities, which improves parallelism.

Simulation

